ARCH Portal

User Guide

A role-based reference

User - PI - Staff - Superuser - Admin

Johns Hopkins University

IT@JHU Research Computing

Advanced Research Computing at Hopkins (ARCH)

CLOACK Stack v.0.3

Table of Contents

Find your role

ARCH Portal (CLOACK Stack) serves five distinct audiences. Each role guide is self-contained for that audience and links into the deep references when you need more.

Who are you?

If you are...	...read	...in one line
A researcher submitting jobs (JHED, `ext-*`, or `ssci-*`)	[`user.md`](user.md)	How to log in, submit `sbatch`, check `sbalance`, read your usage
A PI managing a project, its allocations, and billing	[`pi.md`](pi.md)	How to request a paid allocation, add users, set weekly caps, read the billing report
Helpdesk staff handling tickets	[`staff.md`](staff.md)	How to triage, route by skill, escalate, hit FQR / SLA, run analytics
A ColdFront / Django **superuser** (cluster ops + identity)	[`superuser.md`](superuser.md)	How to approve allocations, drain nodes, reset OTP, rotate the `/admin/` password
A cluster / DevOps **admin** running `start.sh` and Ansible	[`admin.md`](admin.md)	How to build, publish, promote, and deploy the 4-stage pipeline

Not sure which one applies? The roles are not mutually exclusive a PI may also be a user; a superuser may also be an admin. Read the role(s) that match what you are doing right now, then follow the cross-links.

What lives where

- In-app help **/user/help/** (all signed-in users) and **/staff/help/** (Helpdesk staff / superusers) are the live, admin-editable surface for short-form operational notes. They overlap with this directory by design: these role guides are the offline / pre-login / printable reference, the in-app pages are the live working surface.

- Deep technical docs **slurm/README.md**, **coldfront/BILLING_SETUP_GUIDE.md**, the top-level **README.md**, and **CLOACK.md** cover architecture and the full feature matrix. Each role guide links into them rather than duplicating the content.

- Operator runbooks **docs/ansible-remote-deploy.md** and **docs/break-glass-admin.md** are referenced from the Admin and Superuser guides respectively.

- PDF **docs/CLOACK_User_Guide.pdf** concatenates all 5 role guides with a TOC; the developer-facing **docs/CLOACK_Documentation.pdf** covers the technical architecture instead.

How each role guide is structured

Every role doc follows the same 5 sections so you always know where to look:

1. Who you are + scope — 2-3 sentences clarifying which permissions and realms this role typically holds, and which adjacent roles to read if you might be in the wrong place.
2. Quick start — the 5-10 most common tasks for this role as a copy-pasteable checklist.
3. Reference matrices — the few tables you'll come back to (partitions, ticket statuses, deploy stages, etc.).
4. Troubleshooting — symptom -> likely cause -> fix, indexed by error message.
5. Where to go next — links to the deep docs (**slurm/README.md**, **CLOACK.md**, etc.) and to the in-app help.

How this directory is maintained

- Hand-authored Markdown lives in **docs/roles/*.md**.
- HTML mirrors in **docs/roles/*.html** are regenerated by **images/generate_docs_html.py** (run via the **/docs** skill).
- PDF in **docs/CLOACK_User_Guide.pdf** is regenerated by **python3 .claude/skills/pdf/generate_cloack_pdf.py mode users**.
- The **/cloack** skill picks all of this up automatically and pushes to the public GitHub Pages mirror at **cloack.github.io/jhu-arch** (after the Phase 0 GitHub org transfer lands).

User -- Researcher running jobs

Who you are + scope

You are a researcher with an ARCH Portal account: a JHU faculty / staff / student authenticating with JHED, an external collaborator with an **ext-*** username, or a Schmidt Sciences member with an** ssci-*** username. Your day-to-day is submitting Slurm jobs, checking quotas and bills, and managing the data in your own **/home/** and **/scratch/**.

If you also manage allocations for a project (request a new paid allocation, set Weekly Caps, add other users), read **pi.md** too.
If you are a member of the helpdesk team handling tickets, see **staff.md** instead.

Quick start

1. First-time login

Username pattern	Login URL	How you authenticate
JHED ID (e.g. `rdesouz4`)	the JHU portal card	Entra Federation -> MFA from JHU (no separate password)
`ext-jsmith`	the JHU portal card	LDAP password (set during registration) + TOTP (Keycloak account console)
`ssci-jsmith`	the Schmidt portal card	LDAP password (set during registration) + TOTP (Keycloak account console)

If you do not have an account yet:

- JHED users no registration form; just sign in via JHU SSO and the account is created on first login.
- **ext-*** users open** /user/jhu/register/ **on the portal, fill in** the form (the username must start with **ext-**), then wait for your PI to add you to their project / allocation.
- **ssci-*** users same flow but on** /user/schmidt/register/**, and** the username must start with **ssci-**.

After your first successful login a banner at the top of every page prompts you to enrol TOTP in the Keycloak account console. Click the banner link, scan the QR code with Google Authenticator / Authy / Microsoft Authenticator, and you are done. The banner disappears within 30 minutes once the periodic sync picks up the new credential.

2. SSH to a login node

```
ssh <username>@<login-node-host>
```

Authentication on login nodes uses the same identity as the portal:

- JHED browser-based Device Flow opens automatically; approve

in your browser, complete MFA in Entra, the SSH session unblocks.

- **ext-*** /** ssci-*** / **cn=admin** terminal asks for your LDAP

password and then a TOTP code (2 PAM prompts via **pam_kc_ssh.so**).

3. Submit your first job

```
sbatch --partition=med --account=<your-account> --time=00:30:00 \
  --wrap='hostname && date'
```

The Slurm CLI Lua filter prompts you to confirm the cost estimate

before the job is admitted. Press **y** to confirm or **n** to abort.

For longer jobs, use a script:

```
#!/bin/bash
#SBATCH --partition=high
#SBATCH --account=rdesouz4_proj1
#SBATCH --time=04:00:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --job-name=myrun
#SBATCH --output=/home/jhu/rdesouz4/logs/%x-%j.out

module load anaconda
python my_analysis.py
```

4. Check your usage and balance

```
squeue -u $USER          # currently queued or running jobs
sacct -u $USER --starttime=now-7days   # last week's accounting
sbalance                 # core-hours used vs the allocation cap
sbalance --week          # weekly $ cap remaining (paid allocations)
```

sbalance reads from ColdFront's allocation attributes the same

numbers the portal's Allocation Detail page shows.

5. Manage data

Path	Quota	Backup	Cost

`/home/<aff>/<username>`	Small, per-user	Yes (snapshot)	Included
`/home/<aff>/<username>/scratch_<pi>`	Symlink into PI's scratch -- same quota as `/scratch`	No	Included
`/scratch/<aff>/<pi>`	Set per allocation (`storage_tb`)	Optional (`backup_tb`)	`$0.025 / TB / month` storage + `$0.015 / TB / month` backup
`/scratch/<aff>/<pi>/<project>`	Same quota as parent	Inherits parent	Same

Storage and backup are billed per-month regardless of whether you run jobs. Storage charges show up on your PI's billing report under "Storage" / "Backup" lines.

Reference matrices

Slurm partitions

Partition	Nodes	Max wall	Default?	When to use
`scavenger`	`cn-01`	3 days	**Yes**	Free, preemptible. Default partition. Anything you can re-run if killed.
`low`	`cn-01`	3 days	No	Paid, lowest CPU price. Single-node jobs that tolerate slower scheduling.
`med`	`cn-01`, `cn-02`	3 days	No	Paid, mid-tier CPU price. 2-node jobs.
`high`	`cn-02`, `cn-03`	3 days	No	Paid, highest CPU price. Fast scheduling, larger memory nodes.
`premium`	`cn-01`, `cn-02`, `cn-03`	7 days	No	Paid, 7-day wall, all CPU nodes. No `interactive` QoS.
`a100`	`gpu-01`	1 day	No	Paid GPU partition (NVIDIA A100).
`l40s`, `h100`, `h200`	varies	varies	No	Paid GPU partitions per hardware generation.

QoS (job class)

QoS	Priority	Max wall	Notes
`scavenger`	low	3 days	Free. Job is preempted if a paid job needs the resource.
`interactive`	1000	4 hours	Higher priority, capped 4 h. Excluded from `premium`.
`normal`	normal	per partition	Default for paid allocations.
`premium`	high	per partition	Higher priority on `premium` partition.
`ssci`	normal	per partition	Free QoS for `ssci-*` accounts on paid partitions (Schmidt Sciences funded).

Cost (per the live rates.lua shipped in the cluster)

Cost is partition-specific, not global. The numbers below match the Service Center 2026-2034 rate schedule loaded on the dev cluster (see **slurm/conf/rates.lua** for the source of truth; it is auto-generated by **coldfront export_rates_lua** from the **BillingRate** table).

Partition	CPU rate (per SU = core-hour)	GPU rate (per GPU-hour)
`scavenger`	**free** ($0)	n/a
`low`	**$0.0056**	n/a
`med`	**$0.0062**	n/a

`high`	**$0.0107**	n/a
`premium`	**$0.0107**	n/a
`a100`	$0 (GPU billing only)	**$0.5550**
`l40s`, `h100`, `h200`	not yet listed in `rates.lua` -- check with staff before submitting	varies

Storage and backup are flat, per-TB-per-month:

Item	Rate
Storage	**$0.025 / TB / month**
Backup	**$0.015 / TB / month**

Storage and backup are billed every month regardless of whether you run any jobs.

GPU-only billing rule when a job uses any GPU, only its GPU charges go into the bill total; the CPU hours of that job are reported for transparency but excluded from the amount due. Pure-CPU jobs charge CPU as expected.

Practical sense check a typical 8-core, 24 h job on **med** costs **8 cores * 24 h * $0.0062 = $1.19**. A 24 h **a100** job (1 GPU) costs **24 * $0.5550 = $13.32**. Numbers are intentionally small in **Service Center** mode your PI's Weekly Cap protects you from runaways well before the bill bites.

CLI cheat-sheet

Command	Use
`sinfo`	Show partition / node status.
`squeue`	List queued + running jobs (use `-u $USER` to filter).
`sbatch <script>`	Submit a batch job.
`srun --pty bash`	Interactive shell on a compute node.
`scancel <jobid>`	Cancel one of your jobs.
`sacct --starttime=...`	Past job accounting (state, exit code, hours).
`sbalance`	ColdFront-side usage vs allocation cap.
`sbalance --week`	Weekly $ cap remaining for paid allocations.
`seff <jobid>`	Efficiency report after a job finishes.
`stree --full`	Visualise the Slurm account hierarchy you sit under.
`sqos`, `sassoc`, `sfeatures`	ARCH-specific helpers (QoS overview, your associations, partition features).

Troubleshooting

Symptom	Likely cause	Fix
`Permission denied (publickey)` on SSH	Public-key auth disabled by default; you must complete the OTP flow	Re-SSH and follow the terminal prompts (password + TOTP, or browser Device Flow) -- see [`docs/break-glass-admin.md`](../break-glass-admin.md) for the model

Banner says "Activate TOTP" but I already enrolled	Cache up to 30 min -- the `sync_totp_status` task polls Keycloak periodically	Wait 30 min, or hit `/user/user-profile/` to trigger a refresh; if it persists open a helpdesk ticket
`sbatch` error: AssocMaxCpuMinutesPer...	Your weekly $ cap or core-hour cap is exhausted on that allocation	`sbalance --week` to confirm; switch to `scavenger` (free), or ask your PI to raise the Weekly Cap
`sbatch` error: Invalid account or account/partition	The account you passed (e.g. `rdesouz4_proj1`) has no QoS that admits jobs on that partition	Use `sassoc` to see which `account x partition` pairs you can submit to
Job stuck in PD with reason `QOSGrpBillingMinutes`	Your account's Weekly Cap on `billing` TRES was reached	`sbalance --week` to confirm; switch partitions / wait for the weekly window to reset / ask the PI to raise the cap
Job rejected with "accept_cost not set"	A non-CLI submission path (e.g. OnDemand custom app, slurmrestd direct) did not inject `comment=accept_cost`	Re-submit via `sbatch` / `srun` from the login node, or set `--comment=accept_cost` explicitly
Storage charge appears on bill with 0 hours run	Storage and backup are charged monthly regardless of compute activity	This is by design; see the "Storage / Backup billing without jobs" note above
GPU job's CPU hours appear in the report but not in the total	GPU-only billing rule (this page's Cost table)	This is by design -- GPU jobs charge GPU only; CPU hours are shown for transparency
`Permission denied` editing `/scratch/jhu/myproject`	The directory belongs to the PI, not you	Ask your PI to add you to the project / allocation in ColdFront

Where to go next

- **/user/help/** in-app help with live updates.
- **slurm/README.md** full Slurm reference

(cost enforcement, hierarchy, weekly cap details, daemons in depth).

- **pi.md** if you also manage a project.
- **README.md** the role index.

PI -- Managing projects, allocations, and billing

Who you are + scope

You are a Principal Investigator with an ARCH Portal account that has the PI flag on your **UserProfile**. You own one or more Projects, request Allocations against them (default scavenger / paid Pay-Per-Use / condo Hardware-Credit), set Weekly Caps, add users to your projects, and read the billing report monthly. Authentication-wise you are a normal user read **user.md** for the login flow and the job-submission basics.

If you also operate the cluster (deploy services, configure Slurm) you are a different role read **admin.md**. If you handle helpdesk tickets you are also Staff see **staff.md**.

Quick start

1. Accept the Terms of Service

Before you can do anything as a PI you must accept the current ToS. You will see a banner at the top of every portal page until you do. Click the banner -> check the box -> submit. ToS acceptance is also required for every user you add to your projects.

You can re-read what you agreed to at any time: on **/user/user-profile/** the Terms of Service row shows **Accepted vX** as a link that opens the exact version you accepted (read-only, with a Print button).

2. Create a project and request an allocation

```
/project/create/                    -> create a new project
/project/<id>/                      -> project detail page
  Click "Request Resource Allocation"  -> choose a resource (Cluster)
```

A project with empty / **default** abbreviation is the scavenger-only default project: it cannot host paid allocations. To request a paid allocation, give the project a real abbreviation (**proj1**, **alpha**, etc.) when you create it.

For paid allocations, fill in:

- IO Number your funding source. Allocations without an IO number are treated as no-charge regardless of partition.
- storage_tb / backup_tb if you need storage and/or backup.
- Weekly Cap ($) or Weekly Cap (hours) to bound spend.

3. Add users to a project

```
/project/<id>/ -> "Add Users"
```

Users must already have an ARCH Portal account (registered via **/user/jhu/register/** or **/user/schmidt/register/**) and they must have accepted the current ToS. Once you add them, the Slurm associations provision in real time via the post-save signal they can submit jobs against the allocation immediately.

4. Set or update a Weekly Cap

```
/allocation/<id>/ -> click the Weekly Cap field -> edit -> save
```

- Dollar cap takes precedence over hours cap if both are set.
- One cap is enforced per allocation (so users sharing the allocation share the cap).
- The gauge below the Core Usage gauge shows current spend vs cap.

5. Read the billing report

```
/billing/report/                      -> all-PI view (staff-scoped)
/billing/report-by-pi/                -> per-PI view, your projects only
/user/user-profile/ -> "My Usage & Billing"  -> PDF download
```

Storage and backup are billed monthly regardless of compute activity. GPU jobs charge GPU only their CPU hours appear for transparency but are excluded from the total due (the GPU-only billing rule, sourced in **billing_views.calculate_allocation_charges**).

Reference matrices

Allocation lifecycle

State	Set by	What you can do

New	PI on request	Edit IO / storage / backup; cancel before review
Pending review	Auto on submit	Cannot edit; ColdFront staff approve / deny
Active	Staff approval	Add users, set Weekly Cap, submit jobs
Renewal Requested	PI	Same as Active, with a renewal note for staff
Expired / Archived	Auto on end_date	Read-only; usage history preserved

Billing model decision table

If...	...pick
Default scavenger PI project (no abbreviation, no IO)	**No Charge** -- auto-set by `_ensure_allocation_billing` signal
Schmidt PI (`ssci-*`)	**No Charge** -- funded by the Schmidt programme
Paid project, IO number, no upfront hardware purchase	**Pay-Per-Use**
Paid project, IO number, you bought hardware credit	**Condo** (Hardware-Credit holder; `credits_used` gated by `billing_type=credit_holder`)

Cost Center setup (multi-IO)

You can pool multiple IO numbers under a single project / allocation:

```
/billing/cost-centers/                  -> manage your Cost Centers
  -> "Add Cost Center"
  -> Link Cost Center to a project       -> ProjectCostCenter M2M
  -> Assign a Cost Center to an allocation -> AllocationCostCenter M2M
```

ColdFront enforces the Slurm gate: a paid JHU allocation without a pay-role Cost Center linked at the project AND allocation level is silently skipped by **slurm_sync** (no Slurm account created).

Weekly Cap: dollars vs hours

Cap	Units	When to use
Dollar cap	`$` (converted to `GrpTRESMins=billing=X` via `$X x 1000 x 60`)	Most allocations. Predictable monthly spend bound.
Hours cap	core-hours	Hardware-Credit / academic allocations where dollars are not the budget unit.

Per partition the dollar value translates differently because of **TRESBillingWeights**. A **med** partition with **CPU=1.0** costs more per core-hour than **low** with **CPU=0.5**.

Troubleshooting

Symptom	Likely cause	Fix
Banner says "Accept Terms of Service" but I already accepted	Old ToS version superseded by a new one	Visit `/user/user-profile/` and accept again; everything you provisioned before stays valid
Adding a user to a project does nothing visible in Slurm	They have not accepted ToS yet -- the post-save signal skips provisioning until they do	Email the user with the ToS link; once they accept, the immediate-provisioning signal fires

"Request Resource Allocation" button is not shown	This project is a `default` project (no abbreviation)	Create a new project with a real abbreviation; default projects are scavenger-only by design
Allocation is Active but jobs fail with `Invalid account`	The Slurm account hasn't been provisioned yet, or the user's ToS gate has not cleared	Wait for the next `sync_slurm` (every 15 min) or trigger it via Admin actions on the allocation
Storage charge is $0 but I set `storage_tb=0.20`	`0.20 TB x $0.025 = $0.005`, which rounds to $0.00	This is expected at small sizes; ratio is correct
Billing report shows nothing for the past month	The "Computing Billing" button on `/billing/report/` was never pressed	This is by design: `SlurmAccountUsagePeriod` rows are written only when staff explicitly clicks Compute Billing
Hardware Credit pool reports 0 hours despite usage	The allocation's `billing_type` is not `credit_holder`	Set `billing_type=credit_holder` on the AllocationBilling row via Django Admin, or via the Project Detail page if you have staff scope

Where to go next

- **/user/help/** in-app help with live updates.
- **coldfront/BILLING_SETUP_GUIDE.md**

full billing reference (rates, periods, invoices, Cost Centers).

- **user.md** login + day-to-day job submission.
- **README.md** the role index.

Staff -- Helpdesk operator handling tickets

Who you are + scope

You are a Helpdesk staff member with the **helpdesk-staff** LDAP group on your account (or, for team leads, **helpdesk-admin**). You triage the queue, work tickets through the skill-based assignment matrix, escalate on FQR / SLA breaches, and read the analytics dashboards. You authenticate the same way every other user does — see **user.md** for the login flow.

If you also approve allocations or run cluster ops, you are also a Superuser — read **superuser.md**. If you author or read tickets without working them, you are a regular **user.md**.

Quick start

1. Triage a new ticket

```
helpdesk -> /helpdesk/                    -> dashboard with open tickets
  -> click ticket -> ticket detail view
  -> Set Priority, Queue, Owner
  -> Assign by skill                      -> use the skill-match panel on the right
```

The ticket view banner shows urgency (SLA / FQR / strike / overdue). The right-hand context sidebar shows the submitter's past tickets, their PI / project / department, GPU / partition references found in the title / description, similar open tickets, and suggested collaborators ranked by skill match.

2. Respond + classify

```
"Send email as <ResponseTemplate>"      -> picks a template
  -> {name}/{your_name}/{issue}/{ticket_number}/{survey_url}/{signature}
    are auto-substituted
  -> choose the email class (FQR / strike 1-3 / LQR)
  -> server-side scrub catches any token you didn't fill in
```

The classification advances the **TicketFollowUpTracker** and feeds the analytics dashboard. Internal notes use **public=False**; customer

emails use **public=True** collaborators are forced to internal-only.

3. Escalate

```
ticket -> "Escalate" action              -> applies the StaffMember.policy
                                          (3-strike, FQR / SLA breach)
                                         -> reassigns to the next staff in
                                           the routing matrix
                                         -> logs the routing decision
                                           for audit
```

/extensions/analytics/routing/ shows the live decision audit trail.

4. Read the analytics

```
/dashboard/                              -> 3 tabs:
                                            * Department / Project
                                            * PI
                                            * Resource (GPU / CPU / Storage / Backup)
/reports/                                -> 8 stock helpdesk reports rendered
                                            inline as Chart.js bar / line charts
/extensions/skill-matrix/                -> skills x categories grid
/extensions/calendar/                    -> workload calendar
/extensions/schedule/                    -> engineer x day x slot grid +
                                            Time Allocations
```

5. Run a public feedback ticket through the Feedback queue

Tickets created via the public submit form with a category whose **show_on_submit=True** are auto-routed to the Feedback & Suggestions queue (slug **feedback**, owner = **ARCH_SUPERUSERS[0]**). They do not pollute General Support review them on a separate cadence.

Reference matrices

Ticket statuses

ID	Status	Open?	Notes
1	Open	Yes	New, untouched.
2	Reopened	Yes	Customer or staff reopened after Resolved / Closed.
3	Resolved	No	Customer can still reply.
4	Closed	No	Locked.
5	Duplicate	No	Customer redirected to canonical ticket.
6	**Archived**	**No**	**ARCH Portal addition.** Resolved / Closed -> Archived. Single exit: Archived -> Reopened.

Priority + SLA windows

Priority	First Quality Response (FQR)	SLA
1 -- Critical	30 minutes	4 hours
2 -- High	2 hours	1 business day
3 -- Normal	1 business day	3 business days
4 -- Low	2 business days	5 business days
5 -- Very Low	best-effort	best-effort

Breaches trigger escalation per **StaffMember.policy** (3-strike rule shipped by default).

Skill matrix routing

Field	What it does
`StaffSkill(technology_domain, ticket_category)`	A staff's competence per (domain, category) pair
`cognitive_process`	Bloom's taxonomy 1-6; **SME threshold** is the singleton `HelpdeskSMEConfig.cognitive_threshold` (default 6 = Evaluation)
`is_mentor`	Admin override per (staff, category); promotes a staff to SME regardless of `cognitive_process`
`TicketCategory.show_on_submit`	Surface this category on the public submit form
`TicketCategory.default_queue`	Auto-route the ticket to a specific queue (e.g. `feedback`)
`TicketCategory.default_assignee`	Auto-route to a product owner

Helpdesk extensions

URL	What it is
`/extensions/skill-matrix/`	Skills x categories grid; admin-editable
`/extensions/calendar/`	Workload calendar (assigned hours per staff per week)
`/extensions/schedule/`	Engineer x day x 30-min-slot grid + Time Allocations
`/extensions/schedule/request/`	Staff submits a weekly allocation request
`/extensions/schedule/my-requests/`	Staff's own request history
`/extensions/schedule/admin/requests/`	Admin reviews + approves / denies requests
`/extensions/deletion-requests/`	Superuser queue for justified ticket deletions
`/extensions/trello/`	Trello browser (Kanban / Cards / Lists / Gantt)
`/extensions/analytics/routing/`	Routing decision audit dashboard

Troubleshooting

Symptom	Likely cause	Fix
New status 6 (Archived) does not show on a tab I expected	Archived is not in `OPEN_STATUSES = (1, 2)`	Archived is a closed status by design -- it shows on the All / Closed tabs only
Skill matrix shows 0 mentors despite recent edits	`hd_sme_threshold` cache TTL = 5 min	Wait, or `cache.delete("hd_sme_threshold")` from the Django shell
Customer email goes out with `{name}` not substituted	The classification dropdown was bypassed (typed directly into the editor)	Pre-save scrub catches this and re-renders the template; review the ticket and resend if needed
Public feedback ticket routed to General Support	The category does NOT have `default_queue` set	Edit the `TicketCategory` in Django Admin and set `default_queue=Feedback`
Collaborator cannot add a comment	Server-side guard: collaborator can only add `public=False` notes	This is by design; only the owner sends customer emails

Reports page is slow	8 sub-reports rendered inline as Chart.js	The two open-by-default cards (Queue-by-Status, User-by-Status) are the hot path; collapse the rest if needed

Where to go next

- **/staff/help/** in-app help (Staff Help Page).
- **/extensions/** admin UI for skill matrix, schedule grid, analytics, deletion-request queue.
- **superuser.md** if you also approve allocations or run cluster ops.
- **README.md** the role index.

Superuser -- ColdFront and Django admin

Who you are + scope

You have **is_superuser=True** on your Django **User**. You typically also belong to the LDAP **cn=admin** group (cluster sudoers) and the **helpdesk-admin** group (helpdesk privileges). You approve / deny allocations, drain or resume Slurm nodes, reset stale OTP credentials, rotate the local **/admin/** break-glass password, and read the qcluster audit log for every async task.

You typically also operate the cluster (deploy via **start.sh**, manage Ansible bundles) for that side read **admin.md**. For helpdesk-side work see **staff.md**. You authenticate via Entra Federation if you are JHED; if you are an **ext-*** user you use** TOTP via the Keycloak account console (same as every other regular **user.md**).

Quick start

1. Approve / deny an allocation request

```
/admin/allocation/allocation/?status__name=New
  -> open the allocation
  -> review IO, storage, backup, Weekly Cap
  -> action "Approve" or "Deny" (deny requires a reason)
```

Approval triggers **on_allocation_activated**, which provisions the Slurm account hierarchy via REST in real time. ToS gate still applies: PI association is created only if they have accepted the current ToS.

2. Drain / resume a Slurm node

```
/admin/arch_sync/slurmnode/
  -> select node -> action "Drain selected nodes" / "Resume"
  -> reason is logged on the SlurmNode row
```

Behind the scenes this calls **slurmrestd** (or **scontrol** as fallback) on the right cluster. The action is also surfaced on the live

/cluster// page for staff with the cluster in their scope.

3. Reset a stale OTP credential

```
docker exec keycloak bash -c '/opt/keycloak/bin/kcadm.sh ...'
```

See the recipe in **keycloak/README.md**

"Clear a stale OTP credential" it requires:

- **kcadm config credentials** once per session (writes **/tmp/kcadm.config**).
- **kcadm get users -q username=$USER** to fetch the user id.
- **kcadm delete users/$UID/credentials/** to drop the OTP row.

Alternative force the user to re-enrol on their next login:

```
kcadm update users/$UID --config /tmp/kcadm.config -r jhu \
  -s 'requiredActions=["CONFIGURE_TOTP"]'
```

4. Rotate the break-glass /admin/ password

```
bash scripts/rotate-admin-password.sh
```

Generates a 32-char password, applies it via **manage.py changepassword** inside the coldfront container, appends an entry to the gitignored audit log, and prints the password once for paste into the team vault. See **docs/break-glass-admin.md** for the full runbook (vault location, quarterly rotation cadence, **admin_login_alert_task** alert wiring).

5. Read the qcluster audit trail

```
/admin/django_q/task/
/admin/?app=admin&model=logentry   -> Home -> Administration -> Log entries
```

Every **async_task()** call uses the **task_completion_hook** so the admin LogEntry table records when each scheduled task ran, success or failure, duration, and result snippet.

Reference matrices

Scheduled tasks (django-q)

Task	Schedule	Purpose
`sync-ldap-daily`	02:00 daily	Full LDAP sync (PIs, members, groups, home dirs).
`sync-slurm`	every 15 min	Safety net for the signal-based real-time provisioning.
`collect-sacct-periodic`	every 15 min	Pull completed jobs from sacct into `SlurmJob`.
`check-core-hours-reset`	01:00 daily	Trigger scheduled period resets on allocations.
`cleanup-task-queue`	hourly :30	Purge stuck / old qcluster tasks.
`sync-totp-status`	every 30 min	Mirror Keycloak TOTP enrolment state into `UserProfile.totp_enrolled`.
`refresh-slurm-jwt`	every 6 h (:17)	Re-issue the slurmrestd JWT to `/etc/slurm/slurm_jwt.token`.
`sync-bare-metal-topology`	every 10 min	`import_slurm_cluster --update` per bare-metal cluster -- Service Health reflects newly-added nodes within 10 min.

LDAP privilege groups (per affiliation tree)

Group	UID/GID	What it grants
`cn=admin,ou=Groups,ou=jhu`	reserved	Cluster sudoers; `sudo` on every Slurm node. Also auto-imports as Django `is_staff=True` via `seed_initial_data.sync_admin_group_to_is_staff`.
`cn=helpdesk-admin,ou=Groups,ou=jhu`	per affiliation	Helpdesk admin (manage queues, change ticket status, edit categories).
`cn=helpdesk-staff,ou=Groups,ou=jhu`	per affiliation	Helpdesk staff (work tickets, classify, reply).
Cluster (LDAP admin) Django Group	n/a	Mirrors `cn=admin` membership for the Django side. Manage at `/admin/auth/group/`.
`ARCH_SUPERUSERS` env var	n/a	Comma-separated usernames auto-promoted to `is_superuser=True` when they land in `cn=admin`.

Cluster Scope + Affiliation Scope (delegated staff admin)

Scope	M2M target	Controls
Cluster Scope	`Resource` (type Cluster)	Drain / resume nodes, view cluster detail, regenerate the per-cluster Ansible bundle
Affiliation Scope	`Affiliation`	Visibility into users, PIs, projects, allocations of those affiliations only (Manage Users page, Promote PI, scoped list views)
`elevate_to_superuser`	bool	Lets a non-superuser staff approve / deny allocations within their scope

Common Django Admin actions

Path	Action	What it does
`/admin/allocation/allocation/`	Approve / Deny	Approves an allocation, fires real-time Slurm provisioning
`/admin/arch_sync/slurmnode/`	Drain / Resume	Slurm node state via REST
`/admin/arch_sync/slurmcluster/`	Action "Action import from slurmrestd" / "Update from slurmrestd" / "Check drift"	Mirror Slurm topology into ColdFront DB
`/admin/user/userprofile/`	Edit	Set `is_pi`, `is_superuser`, `elevate_to_superuser`, `can_view_billing_stats`
`/admin/auth/group/` -> "Cluster (LDAP admin)"	Add member	Publishes to LDAP `cn=admin` via the `m2m_changed` signal

`/admin/django_q/task/`	View	Live scheduled task table
`/admin/?app=admin&model=logentry`	View	Audit trail (all qcluster + admin actions)

Troubleshooting

Symptom	Likely cause	Fix
Allocation approved but Slurm account not created	ToS not accepted by the PI, OR `on_allocation_activated` raised silently	Check the qcluster Log entries; check `TOSAcceptance` rows; manual fallback: `docker exec coldfront python manage.py sync_slurm`
`/admin/` login alert email not received	`ARCH_ADMIN_ALERT_EMAILS` not set in `.env`, OR recipient flagged the SMTP	Set the env var; check `admin_login_alert_task` LogEntry for the dispatch result
`cn=admin` membership change not visible on login node	SSSD cache; `sss_cache -E` runs at end of every `run_ldap_sync` but TTL = 60 s	Wait one minute, or run `sss_cache -E` manually on the login node
New user created but no UserProfile	The Postgres trigger `tr_auth_user_after_insert_fn` is missing or out-of-date (e.g. `totp_enrolled` column added after trigger)	Re-run `coldfront/init/ldap_sync_postgres_run_it_once.py`; see iter-4 fix `1bffe47`
Scheduled task missing from qcluster	django-q scheduler did not pick up the new schedule (entry added but qcluster not restarted)	`docker restart qcluster`
OTP re-enrol flow does not prompt	User does not have `CONFIGURE_TOTP` in their `requiredActions` and `defaultAction=true` only applies to new users	Use the `kcadm update users` snippet from Quick start §3 to add the required action manually
`helpdesk-admin` LDAP membership changed but Django group did not	The signal `_sync_helpdesk_ldap_groups` only fires on `m2m_changed` for `auth.User_groups` -- the inverse direction does not auto-sync	Run `coldfront python manage.py sync_ldap` to reconcile on the next pass
Allocation in "Pending review" forever	Staff scope does not include the requesting affiliation	Add the affiliation to the staff's `StaffResourceAssignment.affiliations`, OR escalate to a superuser

Where to go next

- **/admin/** Django Admin (Home -> Administration -> Log entries for the qcluster task audit trail).
- **docs/break-glass-admin.md** full break-glass **/admin/** rotation runbook (vault, audit log, alerts).
- **keycloak/README.md** kcadm recipes (OTP credentials, theme re-apply, realm settings inspection).
- **/staff/help/** in-app help with operational notes.
- **admin.md** cluster ops side.
- **README.md** the role index.

Admin -- Cluster and DevOps operator

Who you are + scope

You operate the ARCH Portal stack on physical hosts: the dev VM on Proxmox, the edulogin bare-metal validation host, and the production portal bare-metal host. You run ./start.sh deploy / start / publish / promote, you ship images to GHCR, you run **ansible-playbook** for the bare-metal Slurm cluster install, and you own the rotation of the GHCR PATs.

You are the only role that needs read-write access to GHCR. You typically also have **is_superuser=True** in ColdFront for the cluster ops Django side read **superuser.md**. For helpdesk work see **staff.md**. You authenticate the same way every other user does (**user.md**).

Quick start

Five recipes you will run most days. Every command assumes you have already **git clone**'d the repo at the canonical path on whatever host you are on.

1. Local laptop dev (Stage 1)

```
cd ~/arch_jhu_dsai_schmidt
./start.sh deploy dev -y --paid-accounts   # full clean + reseed dev stack
bash scripts/verify-deploy.sh              # confirm 8/8 checks green
```

Use this when you want a clean known-good state. It wipes named volumes and reseeds 4 test users (**ext-staff**, **ext-user**, **ssci-staff**, **ssci-user**, password **123**).

2. Build and publish on the Proxmox VM (Stage 2)

```
ssh proxmox-vm
cd ~/jhu-arch
git pull origin dev
./start.sh start dev          # local build, no GHCR
bash scripts/verify-deploy.sh # optional smoke
./start.sh publish            # docker push :dev + :sha-<git-short>
```

Publish refuses with a clear error when the working tree is dirty (the **:sha-*** tag would lie about content). Override with **tag** if you genuinely want a throwaway tag.

3. Promote on edulogin (Stage 3)

```
ssh edulogin
cd ~/jhu-arch
git pull origin dev                          # configs / templates only
./start.sh start --from-ghcr dev             # docker pull :dev, no build
bash scripts/verify-deploy.sh                # mandatory pre-flight
./start.sh promote                           # :dev -> :prod + :rc-<UTC>
# Rollback if a bad promotion slipped through:
./start.sh promote --rollback rc-2026-06-09-1430
```

promote runs **verify-deploy.sh** as a pre-flight gate by default; use **skip-verify** only as an emergency override. Never run ./start.sh deploy from-ghcr it is hard-refused. Stages 3 and 4 must never populate test data (the deploy command's job).

4. Deploy on prod portal (Stage 4)

```
ssh portal
cd ~/jhu-arch
git pull origin dev                          # configs only
./start.sh start --from-ghcr prod            # docker pull :prod, no build
bash scripts/verify-deploy.sh
./start.sh ldap-reinit                       # only if LDAP needs re-seed
```

Portal can run Slurm via Docker (pull the slurm images) or via bare-metal Ansible (see playbook recipes below). The compose overlay controls which one is active.

5. Bare-metal Slurm via Ansible

```
cd ~/jhu-arch
./start.sh slurm ansible all                # regenerate ansible/ tree
ansible-playbook -i ansible/clusters/<name>/inventory.ini \
  ansible/slurm-server/playbook.yml         # slurmctld + slurmdbd + slurmrestd
ansible-playbook -i ansible/clusters/<name>/inventory.ini \
  ansible/slurm-client/playbook.yml         # slurmd on every compute node
ansible-playbook -i ansible/clusters/<name>/inventory.ini \
  ansible/slurm-verify/playbook.yml         # read-only pre-flight (no changes)
```

slurm-verify is safe to run any time and reports a 14-check matrix across OS, UIDs, munge, Slurm version, SPANK support, ports, SSSD TTL, and PAM stack.

Reference matrices

4-stage deploy pipeline

Stage	Host	What you run	Result	Image flow
1 Laptop coding	macOS arm64	`./start.sh deploy dev -y --paid-accounts`	Full local dev stack with test data	`arch/<svc>` built locally
1->**2**	Laptop	`git push origin dev`	Source code is on GitHub	--
2 dev VM build	Proxmox Linux amd64	`git pull origin dev`	Code on the builder host	--
2	Proxmox	`./start.sh start dev`	Local build matching prod	`arch/<svc>` built locally on amd64
2	Proxmox	`./start.sh publish`	`:dev` + `:sha-*` pushed to GHCR	`ghcr.io/jhu-arch/cloack/<svc>:dev` + `:sha-<git-short>`
3 edulogin bare-metal	Linux amd64	`git pull origin dev`	Configs / templates on edulogin	--
3	edulogin	`./start.sh start --from-ghcr dev`	Stack runs the `:dev` images, no build	`ghcr.io/...:dev` pulled
3	edulogin	`bash scripts/verify-deploy.sh`	8-check validation result	--
3	edulogin	`./start.sh promote`	`:dev` retagged -> `:prod` + `:rc-<UTC>`	`ghcr.io/...:prod` + `:rc-*` pushed
4 portal bare-metal	Linux amd64	`git pull origin dev`	Configs on portal	--
4	portal	`./start.sh start --from-ghcr prod`	Stack runs the `:prod` images	`ghcr.io/...:prod` pulled
4	portal	`./start.sh ldap-reinit` (manual)	LDAP re-seeded if needed	--

Direction of artefacts:

```
git origin/dev                          GHCR registry
       ?                                    ?
laptop --push--> origin            Stage 2 --push :dev, :sha-*-> GHCR
       ?                                    ?
proxmox --pull--> origin           Stage 3 --pull :dev, push :prod, :rc-*-> GHCR
       ?                                    ?
edulogin --pull--> origin          Stage 4 --pull :prod
       ?                                    ?
portal --pull--> origin            (portal never pushes)
```

GHCR registry namespace and PATs

Stage	PAT scope	Operations
1 (laptop)	none	git push only
2 (Proxmox)	`GHCR_BUILDER_PAT` (write)	`docker push :dev` + `:sha-*`
3 (edulogin)	`GHCR_BUILDER_PAT` (write)	`docker pull :dev`, `docker push :prod` + `:rc-*`
4 (portal)	`GHCR_CONSUMER_PAT` (read-only)	`docker pull :prod` only

Registry path: **ghcr.io/jhu-arch/cloack/:**.

Cross-org topology: the repo lives in the **cloack** GitHub org, packages live in the **jhu-arch** org. The two PATs are fine-grained, scoped to the **jhu-arch** org, and SSO-authorized for Johns Hopkins University Enterprise. Rotation cadence: <= 1 year (fine-grained PAT maximum lifetime).

Image inventory (13 runtime -- 7 cloack apps + 6 Slurm services; +cn-rockylinux build-only = 14 published)

Track	Image	Built from	Promoted past `:dev`?
Cloack	`coldfront`, `helpdesk`, `keycloak`, `keycloak-config`, `openldap`, `phpldapadmin`, `postgres`	service's `Dockerfile`	Yes -- Stage 3 + 4
Slurm (base)	`cn-rockylinux`	`slurm/base/Dockerfile`	**No** -- build dep only, never pulled by runtime hosts
Slurm (services)	`slurmctld`, `slurmdbd`, `slurmrestd`, `slurm-munge`, `mariadb`, `slurmd`	`slurm/<svc>/Dockerfile`	Yes -- `promote` retags these **best-effort** (skips any not published, e.g. bare-metal Slurm). `slurmd` is the login-node image too, so it promotes even when compute workers run bare-metal.

Note that **cn-rockylinux** is consumed at build time by the other Slurm images (each does a **FROM arch/cn-rockylinux**). Once a Slurm service image is pushed, the cn-rockylinux layers travel inside it runtime hosts (Stage 3 + 4) never pull cn-rockylinux as a standalone image.

Stage 4 dual-mode Slurm

Mode	Slurm runs as	What portal pulls	When to use
A -- Docker Slurm	containers	7 cloack apps + 6 Slurm services	All-Docker portal, smaller HPC scale
B -- Bare-metal Slurm	services on real hosts via Ansible roles	7 cloack apps only	Real HPC scale, dedicated compute nodes, kernel-level features
A2 -- Slurm current target	core Slurm containerized; **compute workers excluded**	7 cloack apps + 6 Slurm services (workers not started; login-node `slurmd` still runs)	Containerized controller + login + `slurmdbd`/`slurmrestd`/`mariadb`, compute nodes added later (bare-metal or Docker)

In Mode B the operator runs **ansible/slurm-server/playbook.yml** and **ansible/slurm-client/playbook.yml** against the cluster inventory. **cluster_manager.write_compose_overlay** honours **CLOACK_FROM_GHCR=1** to rewrite Slurm image refs to **ghcr.io/...:prod** in Mode A.

start.sh subcommands by stage

Subcommand	Stage(s)	What it does
`init [dev]`	1, 2	Bootstrap user/group/dirs/`.env`. Run once before first start.
`start [dev]`	1, 2	Build (or skip build) and start managed services.
`start --from-ghcr [dev|prod]`	3, 4	Pull from GHCR, skip build. Refuses if `docker-compose-ghcr.yml` is missing.
`deploy [dev] -y [--paid-accounts]`	1, 2	Destructive: clean + init + start + populate test data. Hard-refused with `--from-ghcr`.
`publish [--tag <tag>] [--check]`	2	Retag local `arch/<svc>` as `ghcr.io/...:<tag>` + `:sha-<short>` and push. Refuses dirty working tree.
`promote [--from <tag>] [--rc <label>] [--check] [--rollback <rc-tag>] [--skip-verify]`	3	Pull `:dev`, run `verify-deploy.sh`, retag + push `:prod` + `:rc-<UTC>`.
`ldap-init [dev|prod]`	3, 4	Re-initialise OpenLDAP from migration LDIF, force sync with ColdFront.

`slurm {create\	start\	stop\	restart\	status\	test-jobs\	ansible\	prod-sim}`	varies	Per-cluster Slurm operations. `ansible all` regenerates the bare-metal scaffold.
`helpdesk {start\	populate\	restart\	stop}`	1, 2	Helpdesk container lifecycle.				
`doctor`	any	Read-only validation of `.env` <-> realm config <-> TLS cert <-> Host header drift.							

Ansible playbooks

Playbook	What it does	When to run
`ansible/slurm-server/playbook.yml`	Compiles Slurm from source, installs slurmctld / slurmdbd / slurmrestd on the controller groups	Bare-metal install or major Slurm version bump
`ansible/slurm-client/playbook.yml`	Same build stack plus slurmd on every compute node and login node	New compute node, or after `slurm-server`
`ansible/slurm-verify/playbook.yml`	Read-only 14-check pre-flight (OS, UIDs, munge, Slurm version, SPANK, ports, SSSD TTL, PAM stack). No changes applied.	Any time you want a confidence check
`ansible/sssd-flush/playbook.yml`	Runs `sss_cache -E` on every login node	`sync_ldap` already triggers this per cluster; manual run useful after a large LDIF import
`cloack-deploy` RPM	Thin packaging of the Ansible tree + a `cloack-deploy` CLI wrapper	Sites that prefer an RPM-driven flow over `ansible-playbook` directly

Environment variables you will set on Stage 3 / Stage 4 hosts

Phase 2 (commit **a525237**) added multi-FQDN reverse-proxy support. The following env vars are opt-in; leave them empty in single-FQDN dev to keep current behaviour.

Variable	Purpose
`ARCH_PUBLIC_HOST`	The portal FQDN (`portal.arch.jhu.edu`). Used everywhere.
`KEYCLOAK_HOSTNAME_JHU`	JHU realm FQDN (`auth.arch.jhu.edu`). Empty -> single-FQDN.
`KEYCLOAK_HOSTNAME_SCHMIDT`	Schmidt realm FQDN. Empty -> single-FQDN.
`KC_HOSTNAME`	Empty in multi-FQDN; Keycloak uses inbound Host instead.
`KC_HOSTNAME_BACKCHANNEL_DYNAMIC`	`true` in multi-FQDN deploys (trust `X-Forwarded-Host`).
`HELPDESK_SCRIPT_NAME`	`/helpdesk` in prod to route `portal.arch.jhu.edu/helpdesk` to the helpdesk container. Empty -> no prefix.
`CLOACK_IMAGE_TAG`	Auto-set by `--from-ghcr`; override only for rollback (`sha-abc123` or `rc-YYYY-MM-DD-HHMM`).
`CLOACK_FROM_GHCR`	Auto-set by `--from-ghcr`; do not set by hand.

Troubleshooting

Symptom	Likely cause	Fix

`keycloak-config Error... exit 5` on first deploy	`jq` could not index into an error response on `/auth/flows` (realm not imported yet)	Already fixed in commit `cd35aad`; if reappears, check the `_GHCR_IMAGE_REWRITE_RE` regex hasn't been broken by a new image name pattern	
`keycloak-config Error... exit 3` on first deploy	`jq` tried `ascii_downcase` on a null `displayName` (Keycloak default executions list)	Fixed in `8443758` -- re-pull the latest dev if you see this	
`curl: URL malformed` in configure-keycloak.sh	Copied flow has a literal space in its sub-flow alias (`passwordless-browser forms`)	Fixed in `d0dfafe` -- `${forms_sub_alias// /%20}` URL-encodes the space	
`IntegrityError: null value in column "totp_enrolled"`	Postgres trigger `tr_auth_user_after_insert_fn` was created before the `totp_enrolled` column existed	Fixed in `1bffe47` -- drop the trigger and re-run `ldap_sync_postgres_run_it_once.py`	
Themes do not apply on first deploy (default Keycloak look)	`configure_realm_themes` ran before `import_keycloak_providers` so PUT returned 404 silently	Fixed in `b7efa22` -- re-runs the idempotent block after realms are imported	
`publish: working tree is dirty`	You have uncommitted source changes	Commit them, or use `./start.sh publish --tag wip-<something>` to bypass the SHA-tag promise	
`deploy --from-ghcr is not allowed`	Stages 3 and 4 must never populate test data	Use `./start.sh start --from-ghcr <tag>` plus `./start.sh ldap-reinit` instead	
`publish: no GHCR auth in ~/.docker/config.json`	Builder PAT not loaded	`echo $GHCR_BUILDER_PAT \`	docker login ghcr.io -u <user> --password-stdin`
`--from-ghcr: docker-compose-ghcr.yml not found`	Phase 0 file missing in the repo on this host	Pull latest dev (`52b709f` introduced the file)	
`pgrep -f "start.sh deploy"` loop never exits	The waiter's own command line contains the pattern, deadlock	Track `$DEPLOY_PID` directly with `wait $DEPLOY_PID`; see memory `feedback_pgrep_self_match_deadlock`	
Slurm overlay still emits `arch/<svc>` after `--from-ghcr`	`CLOACK_FROM_GHCR` was not exported when `write_compose_overlay` ran	Re-run `./start.sh slurm create <name>` after exporting the var	
`configure_realm_frontend_urls` did nothing visible	`KEYCLOAK_HOSTNAME_JHU` / `_SCHMIDT` are empty	Set them in `.env` for multi-FQDN, or leave them empty for single-FQDN -- the function is a no-op then by design	

Where to go next

- **docs/ansible-remote-deploy.md** full bare-metal Ansible deploy guide, RPM install, idempotent re-runs.

- **docs/break-glass-admin.md** **/admin/** break-glass rotation (vault, audit log, alerts).

- **slurm/README.md** full Slurm config reference (partitions, QOS, weekly cap, CLI tools).

- **slurm/ANSIBLE.md** bare-metal Slurm install via the **cloack-deploy** RPM path.

- **docs/deploy-pipeline-4-stages.md** the design plan behind the matrix on this page; read when the model needs to evolve.

- **superuser.md** ColdFront / Django admin side

(drain nodes, approve allocations, qcluster audit, OTP resets).

- **README.md** the role index.